指示フロー図ダイアログ UI設計書

メタデータ

項目	内容
コンポーネントID	InstructionFlowDialog
物理ファイル名	InstructionFlowDialog.tsx
ファイルパス	frontend/src/components/ExperimentWorker/InstructionFlowDialog.tsx
カテゴリ	実験作業者（Experiment Worker）
権限	experiment_worker
作成日	2026-01-19
最終更新日	2026-03-11

---

1. 概要

1.1 目的

指示確認タブ専用のフロー図表示ダイアログです。選択中のメソッドのPetri Netフロー図をドラッグ&リサイズ可能なフローティングウィンドウで表示します。

1.2 主要機能

- MAIMLファイルからフロー図を読み込んで表示
- ドラッグによるウィンドウ移動
- リサイズによるウィンドウサイズ変更
- 最小化機能（タイトルバーのみ表示）
- 閉じる機能
- サブネットビューアの起動（トランジションクリック時）
- Z-index管理による複数ウィンドウの重ね順制御

1.3 使用場所

- InstructionConfirmationTab（指示確認タブ）内
- 「フロー図を表示」ボタン押下時に表示

1.4 他のダイアログとの違い

項目	InstructionFlowDialog	ViewRendererDialog
用途	指示確認タブ専用	汎用的なファイル表示
メソッド選択Tabs	非表示（hideMethodTabs: true）	表示可能
対象メソッド	選択中のメソッドのみ	全メソッド
配置	指示確認タブ内	実験記録タブなど

---

2. コンポーネント構成

2.1 画面イメージ



2.2 レイアウト構造

```
Portal (document.body)
└── Rnd (ドラッグ&リサイズ可能)
    └── ダイアログコンテナ (白背景、角丸、シャドウ)
        ├── タイトルバー (drag-handle, グレー背景)
        │   ├── Text - ファイル名（太字、16px）
        │   └── ボタン群
        │       ├── Button (最小化/展開)
        │       └── Button (閉じる)
        │
        └── コンテンツエリア (最小化時は非表示)
            ├── Spin (ローディング中)
            └── ViewRenderer (フロー図表示)
                └── PetriNetVisualization
                    └── React Flow
```

2.3 使用コンポーネント

Ant Design:

- Spin - ローディング表示
- Button - タイトルバーのボタン
- Typography.Text - タイトル表示
- message - エラーメッセージ表示

Ant Design Icons:

- CloseOutlined - 閉じるボタン
- MinusOutlined - 最小化ボタン
- ExpandOutlined - 展開ボタン

サードパーティ:

- react-rnd (Rnd) - ドラッグ&リサイズ機能
- ReactDOM.createPortal - Portalレンダリング

カスタムコンポーネント:

- ViewRenderer - フロー図レンダラー（hideMethodTabs: true で使用）

---

3. Props定義

3.1 Props一覧

```
interface InstructionFlowDialogProps {
  id: string;                              // ダイアログID（一意識別子）
  file: FileData;                          // 表示するファイル情報
  selectedMethodId: string | null;         // 選択中のメソッドID
  zIndex: number;                          // Z-index（重ね順）
  initialPosition?: { x: number; y: number };    // 初期位置
  initialSize?: { width: number; height: number }; // 初期サイズ
  onClose: (id: string) => void;           // 閉じる時のコールバック
  onFocus: (id: string) => void;           // フォーカス時のコールバック
```

```
  onOpenSubNetViewer?: (subnetData: SubnetData) => void; // サブネット表示
}
```

3.2 Props詳細

Prop名	型	必須	デフォルト	説明
id	string	○	-	ダイアログの一意識別子
file	FileData	○	-	表示するMAIMLファイル情報
selectedMethodId	string | null	○	-	選択中のメソッドID（ViewRendererに渡される）
zIndex	number	○	-	ウィンドウの重ね順（大きいほど手前）
initialPosition	object	×	{ x: 100, y: 100 }	初期表示位置
initialSize	object	×	{ width: 1200, height: 800 }	初期サイズ
onClose	function	○	-	閉じるボタン押下時のコールバック
onFocus	function	○	-	ウィンドウクリック時のコールバック（Z-index更新用）
onOpenSubNetViewer	function	×	-	サブネットビューア起動時のコールバック

3.3 型定義

```
interface FileData {
  id: string;           // ファイルID
  name: string;         // ファイル名
  path: string;         // ファイルパス
  size: number;         // ファイルサイズ
  lastModified: string; // 最終更新日時
  type: string;         // ファイルタイプ（'maiml'など）
  isFolder?: boolean;   // フォルダかどうか
  fileCount?: number;   // フォルダ内のファイル数
  content?: string;     // APIから取得済みのファイルコンテンツ（設定されていればfetchをスキップ）
}

interface SubnetData {
  transitionName: string;  // トランジション名
  maimlData: string;       // MAIMLデータ
```

```
  templateMap: Record<string, { element: Element }>;
  petriNetData: {
    places: Array<{
      id: string;
      data: {
        label?: string;
        stereotype?: string;
        template?: { id: string };
        templateId?: string;
      };
    }>;
    arcs: Array<{
      id: string;
      source: string;
      target: string;
    }>;
  };
}
```

3.4 内部State

```
const [fileContent, setFileContent] = useState<string | null>(null);  // ファイル内容
const [loading, setLoading] = useState(false);                         // ローディング状態
const [isMinimized, setIsMinimized] = useState(false);                // 最小化状態
```

---

4. 主要機能

4.1 ファイル読み込み

```
const loadFileContent = async (file: FileData): Promise<string | null> => {
  try {
    if (file.type === 'maiml') {
      // APIから取得済みのコンテンツが存在する場合はそれを優先使用
      if (file.content) {
        return file.content;
      }

      // フォールバック: モックデータを読み込む
```

```
      const response = await fetch('/data/mock/test_omori11.maiml');

      // HTTPエラーチェック
      if (!response.ok) {
        throw new Error(`Failed to load MAIML file: ${response.status}
${response.statusText}`);
      }

      const content = await response.text();
      return content;
    }

    // その他のファイルタイプはコンテンツなし
    return null;
  } catch (error) {
    // エラー時はメッセージを表示してnullを返す
    message.error(`ファイルの読み込みに失敗しました: ${error instanceof Error ? error.message :
'不明なエラー'}`);
    return null;
  }
};
```

優先順位:

1. `file.content` が設定されている場合 → APIから取得済みコンテンツをそのまま使用
2. `file.content` がない場合 → `/data/mock/test_omori11.maiml` をfetchして使用（後方互換フォールバック）

useEffectで自動読み込み:

- file が変更されたら自動的にファイル内容を読み込み
- マウント解除時にクリーンアップ（isMounted フラグ）

4.2 ドラッグ&リサイズ（react-rnd）

Rndコンポーネントの設定:

```
<Rnd
  default={{
    x: initialPosition.x,
    y: initialPosition.y,
    width: initialSize.width,
    height: isMinimized ? 60 : initialSize.height,
```

```
  }}
  minWidth={800}
  minHeight={isMinimized ? 60 : 600}
  maxWidth={window.innerWidth - 100}
  maxHeight={window.innerHeight - 100}
  bounds="window"
  dragHandleClassName="drag-handle"
  enableResizing={!isMinimized}
  onMouseDown={() => onFocus(id)}
>
```

制約:

- 最小幅: 800px
- 最小高さ: 600px（通常時）、60px（最小化時）
- 最大幅: 画面幅 - 100px
- 最大高さ: 画面高さ - 100px
- 移動範囲: ウィンドウ内（bounds="window"）
- ドラッグハンドル: `.drag-handle` クラスの要素（タイトルバー）
- リサイズ: 最小化時は無効

4.3 最小化機能

```
// 最小化ボタン押下
<Button
  icon={isMinimized ? <ExpandOutlined /> : <MinusOutlined />}
  onClick={() => setIsMinimized(!isMinimized)}
/>

// 高さの制御
height: isMinimized ? 60 : initialSize.height,

// コンテンツの非表示
{!isMinimized && (
  <div>{renderContent()}</div>
)}
```

4.4 Z-index管理

```
// ウィンドウクリック時にフォーカス（Z-indexを最前面に）
onMouseDown={() => onFocus(id)}

// 親コンポーネントでZ-indexを更新
<InstructionFlowDialog
  zIndex={dialog.zIndex}
  onFocus={handleFocusDialog}  // Z-indexを更新して最前面に
/>
```

4.5 Portal レンダリング

```
// document.body に直接レンダリング（モーダル外に配置）
if (typeof document !== 'undefined') {
  return ReactDOM.createPortal(dialogContent, document.body);
}

// SSR時はnullを返す
return null;
```

---

5. データフロー

5.1 初期化フロー

```
1. Props受け取り
   ↓
2. useEffect でファイル読み込み
   ↓
3. loadFileContent(file)
     ├─ file.content が存在する → APIから取得済みコンテンツをそのまま使用
     └─ file.content がない    → fetch('/data/mock/test_omori11.maiml') にフォールバック
   ↓
4. setFileContent(content)
   ↓
5. ViewRenderer にファイル内容を渡す
   ↓
6. PetriNetVisualization でフロー図表示
```

5.2 インタラクションフロー

```
【ドラッグ】
ユーザーがタイトルバーをドラッグ
  ↓
Rnd がウィンドウ位置を更新
  ↓
onFocus(id) でZ-indexを最前面に

【リサイズ】
ユーザーがウィンドウの端をドラッグ
  ↓
Rnd がウィンドウサイズを更新

【最小化】
ユーザーが最小化ボタンをクリック
  ↓
setIsMinimized(true)
  ↓
高さを60pxに変更、コンテンツを非表示

【閉じる】
ユーザーが閉じるボタンをクリック
  ↓
onClose(id)
  ↓
親コンポーネントでダイアログを削除
```

---

6. スタイリング

6.1 Rnd コンポーネント

```
// Rnd コンポーネント自体にも zIndex を設定（Portal レンダリング時の重ね順制御）
<Rnd
  style={{ zIndex: zIndex }}
  ...
>
```

6.1.1 ダイアログコンテナ（Rnd 内側）

```
{
  width: '100%',
  height: '100%',
  backgroundColor: '#fff',
  borderRadius: 8,
  boxShadow: '0 6px 16px 0 rgba(0, 0, 0, 0.08), 0 3px 6px -4px rgba(0, 0, 0, 0.12), 0 9px 28px
8px rgba(0, 0, 0, 0.05)',
  display: 'flex',
  flexDirection: 'column',
  overflow: 'hidden',
  position: 'relative',
  zIndex: zIndex,  // Props から受け取ったZ-index（Rnd 内側コンテナにも設定）
}
```

6.2 タイトルバー

```
{
  padding: '16px 24px',
  borderBottom: isMinimized ? 'none' : '1px solid #f0f0f0',
  cursor: 'move',
  display: 'flex',
  justifyContent: 'space-between',
  alignItems: 'center',
  backgroundColor: '#fafafa',
  borderRadius: isMinimized ? '8px' : '8px 8px 0 0',
  flexShrink: 0,
}
```

特徴:

- `cursor: 'move'` - ドラッグ可能を示す
- 最小化時: ボーダーなし、全体が角丸
- 通常時: 下部ボーダーあり、上部のみ角丸

6.3 コンテンツエリア

```
// id 属性付き（コンテナの一意識別に使用）
<div
  id={`instruction-flow-dialog-container-${id}`}
  style={{
    position: 'relative',
    flex: 1,
    overflow: 'hidden',
  }}
>
```

---

7. イベント定義

7.1 ユーザーイベント

イベント	トリガー	処理
ウィンドウクリック	onMouseDown	onFocus(id) でZ-indexを最前面に
最小化ボタンクリック	onClick	setIsMinimized(!isMinimized)
閉じるボタンクリック	onClick	onClose(id)
トランジションクリック	ViewRenderer経由	onOpenSubNetViewer(subnetData)

7.2 ライフサイクルイベント

イベント	タイミング	処理
マウント時	useEffect初回	ファイル内容を読み込み
file変更時	useEffect	ファイル内容を再読み込み
アンマウント時	useEffect cleanup	isMountedフラグをfalseに

---

8. パフォーマンス考慮事項

8.1 最適化

- **Portal レンダリング**: document.body に直接レンダリングし、親コンポーネントの再レンダリングの影響を軽減
- **isMounted フラグ**: アンマウント後のsetStateを防止
- **条件付きレンダリング**: 最小化時にコンテンツを非表示にしてレンダリングコストを削減

8.2 注意点

- **React Flow の負荷**: PetriNetVisualization内のReact Flowは大規模なグラフで負荷が高い
- **複数ウィンドウ**: 同時に複数のダイアログを開くとメモリ使用量が増加
- **ファイル読み込み**: 毎回fetchするため、キャッシュ機構があるとより効率的

---

9. アクセシビリティ

9.1 対応項目

- ✅ ボタンに適切なtitle属性（「閉じる」「最小化」「展開」）
- ✅ キーボード操作可能（ボタンがフォーカス可能）
- ✅ 視覚的フィードバック（ホバー、フォーカス）

9.2 改善検討事項

- ⚠️ role="dialog" 属性の追加
- ⚠️ aria-label または aria-labelledby でダイアログのラベル付け
- ⚠️ Escapeキーで閉じる機能
- ⚠️ フォーカストラップ（ダイアログ内でTabキーが循環）

---

10. セキュリティ

10.1 セキュリティ考慮事項

- **XSS対策**: Ant Design コンポーネントが自動的にエスケープ処理

- **ファイルパスの検証**: fetch('/data/mock/test_omori11.maiml') は固定パスなので安全
- **Portal のスコープ**: document.body に追加されるため、CSSの影響範囲に注意

---

11. テスト要件

11.1 単体テスト

```
describe('InstructionFlowDialog', () => {
  test('ダイアログが正しくレンダリングされる', () => {
    const { getByText } = render(
      <InstructionFlowDialog
        id="dialog1"
        file={mockFile}
        selectedMethodId="method1"
        zIndex={1000}
        onClose={mockOnClose}
        onFocus={mockOnFocus}
      />
    );
    expect(getByText(mockFile.name)).toBeInTheDocument();
  });

  test('最小化ボタンで最小化できる', () => {
    const { getByTitle, queryByText } = render(
      <InstructionFlowDialog {...defaultProps} />
    );
    const minimizeButton = getByTitle('最小化');
    fireEvent.click(minimizeButton);
    // コンテンツが非表示になることを確認
  });

  test('閉じるボタンでonCloseが呼ばれる', () => {
    const mockOnClose = jest.fn();
    const { getByTitle } = render(
      <InstructionFlowDialog {...defaultProps} onClose={mockOnClose} />
    );
    const closeButton = getByTitle('閉じる');
    fireEvent.click(closeButton);
    expect(mockOnClose).toHaveBeenCalledWith(defaultProps.id);
  });

  test('ウィンドウクリックでonFocusが呼ばれる', () => {
    const mockOnFocus = jest.fn();
```

```
    const { container } = render(
      <InstructionFlowDialog {...defaultProps} onFocus={mockOnFocus} />
    );
    fireEvent.mouseDown(container.firstChild);
    expect(mockOnFocus).toHaveBeenCalledWith(defaultProps.id);
  });
});
```

---

12. 関連ドキュメント

12.1 関連コンポーネント

- InstructionConfirmationTab - このダイアログを表示するタブ
- ViewRenderer - フロー図レンダリングに使用
- SubNetViewer - サブネット表示に使用
- PetriNetVisualization - PetriNet可視化

12.2 関連仕様

- .clinerules/references/ui_policy.md - UI設計ポリシー
- .clinerules/development/frontend.md - フロントエンド規約

---

13. 備考

13.1 設計上の特徴

1. Portal パターン:

- `ReactDOM.createPortal` で document.body に直接レンダリング
- モーダルの外に配置され、親コンポーネントのスタイルやZ-indexの影響を受けない
- 複数のダイアログを独立して管理可能

2. Rndによるドラッグ&リサイズ:

- ユーザーが自由にウィンドウを配置・サイズ変更可能
- bounds="window" でウィンドウ外に出ないように制約
- dragHandleClassName でタイトルバーのみドラッグ可能

3. ViewRendererの再利用:

- `hideMethodTabs: true` で メソッド選択Tabsを非表示
- 既存のViewRendererコンポーネントを活用して実装コストを削減

13.2 ViewRendererDialogとの違い

項目	InstructionFlowDialog	ViewRendererDialog
用途	指示確認タブ専用	汎用的なファイル表示
メソッドTabs	非表示（hideMethodTabs: true）	表示可能
対象	選択中のメソッドのみ	全メソッド
配置場所	指示確認タブ内	実験記録タブなど
ファイル	InstructionFlowDialog.tsx	ViewRendererDialog.tsx

13.3 将来の拡張

- ✅ **複数ダイアログ管理**: すでに対応済み（Z-index管理）
- ⚠️ **ウィンドウ位置の記憶**: LocalStorageに保存して復元
- ⚠️ **スナップ機能**: 画面端に吸着する機能
- ⚠️ **全画面表示**: Maximizeボタンの追加
- ⚠️ **キーボードショートカット**: Ctrl+W で閉じるなど

13.4 技術的な注意点

1. **SSR対応**: `typeof document !== 'undefined'` でSSR時の対応
2. **クリーンアップ**: useEffect内で isMounted フラグによる適切なクリーンアップ
3. **Z-index競合**: 親コンポーネントで適切に Z-index を管理する必要がある
4. **メモリリーク**: アンマウント後のsetStateを防止

13.5 使用例

```
// InstructionConfirmationTab内での使用例
const [openDialogs, setOpenDialogs] = useState<DialogData[]>([]);

// フロー図を表示ボタン押下時
const handleShowFlowDiagram = () => {
  const newDialog: DialogData = {
    id: `instruction-flow-${Date.now()}`,
    file: currentFile,
    zIndex: getNextZIndex(),
  };
  setOpenDialogs([...openDialogs, newDialog]);
};

// レンダリング
{openDialogs.map((dialog) => (
  <InstructionFlowDialog
    key={dialog.id}
    id={dialog.id}
    file={dialog.file}
    selectedMethodId={selectedMethodId}
    zIndex={dialog.zIndex}
    onClose={handleCloseDialog}
    onFocus={handleFocusDialog}
    onOpenSubNetViewer={handleOpenSubNetViewer}
  />
))}
```

---

14. 変更履歴

バージョン	日付	変更内容	担当者
1.1.0	2026-03-11	ソースコードとの差分解消	Cline
		- §3.3 `FileData` 型に `content?: string` フィールドを追加	
		- §4.1 `loadFileContent` のロジックを更新（`file.content` 優先・HTTPエラーチェック・エラーハンドリング追加）	
		- §5.1 初期化フローに `file.content` 分岐を追記	

		- §6.1 `Rnd` コンポーネントへの `style={{ zIndex }}` 設定を追記	
		- §6.3 コンテンツエリアの `id` 属性を追記	
1.0.0	2026-01-19	初版作成	Cline